1 задача: сравнение последовательной и параллельной реализаций множества Мандельброта

0. Условия задачи

Написать параллельную программу для нахождения заданного количества точек множества Мандельброта.

Формат входных данных: Запуск из командной строки ./program nthreads npoints, где nthreads — количество потоков, npoints — количество точек.

Формат выходных данных: Файл формата csv с координатами точек множества Мандельброта.

1. Цель эксперимента

Сравнить последовательную и параллельную реализации по корректности результата и времени вычислений. Для параллельной версии проверить зависимость времени от числа потоков.

2. Окружение эксперимента

Заполнить после выполнения команд:

- Процессор (модель): AMD Ryzen 5 5500U with Radeon Graphics
- Логические процессоры: 12
- Операционная система: Майкрософт Windows 11 Домашняя для одного языка 10.0.26200
- Компилятор: Microsoft (R) C/C++ Optimizing Compiler Version 19.51.36260 for x86
- Архитектура сборки: 64-разрядная
- Флаги сборки последовательной версии: `/O2 /EHsc`
- Флаги сборки параллельной версии: `/O2 /EHsc /openmp`
- Размер входа (`npoints`): 1000000
- Число повторных запусков для каждого варианта:10
- Итоговая метрика: медиана времени по 10 запускам

3. Алгоритмы

Последовательная версия

Программа последовательно перебирает точки квадратной сетки комплексной плоскости. Программа sequential.exe принимает один аргумент: npoints и последовательно обрабатывает все индексы сетки, сохраняет результаты в массив и после вычислений записывает координаты найденных точек в mandelbrot_simple.csv

Параллельная версия

Вычисление каждой точки независимо от остальных, поэтому цикл по индексам сетки распараллелен с помощью OpenMP `parallel for`. Число потоков задаётся аргументом `nthreads`, а `schedule(dynamic)` распределяет итерации динамически между потоками. Программа parallel.exe принимает два аргумента: nthreads и npoints. После завершения параллельных вычислений программа записывает найденные точки в mandelbrot.csv.

Особенность параметра `npoints`

В данной реализации фактическое число проверяемых точек равно sqrt(npoints)^2. Из кода, 33 строка lab1ex1_OpenMP.cpp:

```
int n_side = (int)sqrt((double)npoints);   // целая часть корня
int total  = n_side * n_side;              // реальное число точек
```

Поэтому для тестов имеет смысл брать `npoints`, являющийся полным квадратом, например `1000000` (квадрат 1000 × 1000). Так же CSV-файл содержит только точки, признанные принадлежащими множеству, а не все проверенные точки.

4. Запуск и замер времени

Запуск:

```
sequential.exe 1000000
parallel.exe 4 1000000
```

Результат:

```
Time: 0.438419 s (sequential, npoints=1000000, n_side=1000)
Time: 0.140607 s (nthreads=4, npoints=1000000, n_side=1000)
```

Запуск:

```
fc /N mandelbrot_simple.csv mandelbrot.csv
```

Результат:

```
Сравнение файлов mandelbrot_simple.csv и MANDELBROT.CSV
FC: различия не найдены
```

Это означает, что файлы совпадают побайтно: одинаковы заголовок реальные и мнимые части, координаты, порядок строк и формат чисел. Для приведённого кода именно такой результат ожидается.

Запуск:

```
find /c /v "" mandelbrot_simple.csv
```

Результат:

```
---------- MANDELBROT_SIMPLE.CSV: 167492
```

Запуск:

```
find /c /v "" mandelbrot.csv
```

Результат:

```
---------- MANDELBROT.CSV: 167492
```

Это означает, что файлы содержат одинаковое количество строк, в каждом по 167492. Совпадение количества строк само по себе не доказывает совпадение координат, поэтому основной проверкой является fc (сравнением побайтно).

Повторить каждый запуск 10 раз

Следующие команды PowerShell запускают программу 10 раз и сохраняют её вывод в отдельный файл. Их можно вводить из CMD:

```
powershell -NoProfile -Command "1..10 | ForEach-Object { & .\sequential.exe 1000000 } | Tee-O
powershell -NoProfile -Command "1..10 | ForEach-Object { & .\parallel.exe 1 1000000 } | Tee-O
powershell -NoProfile -Command "1..10 | ForEach-Object { & .\parallel.exe 2 1000000 } | Tee-O
powershell -NoProfile -Command "1..10 | ForEach-Object { & .\parallel.exe 4 1000000 } | Tee-O
powershell -NoProfile -Command "1..10 | ForEach-Object { & .\parallel.exe 8 1000000 } | Tee-O
```

Эти команды корректно передают аргументы каждой программе и сохраняют её текстовый вывод в отдельный файл. Каждый запуск перезаписывает соответствующий CSV-файл, но файлы последовательной и параллельной версий имеют разные имена.

sequential_runs.txt:

```
Time: 0.442892 s (sequential, npoints=1000000, n_side=1000)
Time: 0.444494 s (sequential, npoints=1000000, n_side=1000)
```

```
Time: 0.440474 s (sequential, npoints=1000000, n_side=1000)
Time: 0.442446 s (sequential, npoints=1000000, n_side=1000)
Time: 0.437543 s (sequential, npoints=1000000, n_side=1000)
Time: 0.449801 s (sequential, npoints=1000000, n_side=1000)
Time: 0.446613 s (sequential, npoints=1000000, n_side=1000)
Time: 0.447703 s (sequential, npoints=1000000, n_side=1000)
Time: 0.442716 s (sequential, npoints=1000000, n_side=1000)
Time: 0.441852 s (sequential, npoints=1000000, n_side=1000)
```

Медиана = (0.442716 + 0.442892) / 2 = 0.442804 с

parallel_1_runs.txt:

```
Time: 0.450965 s (nthreads=1, npoints=1000000, n_side=1000)
Time: 0.454839 s (nthreads=1, npoints=1000000, n_side=1000)
Time: 0.452329 s (nthreads=1, npoints=1000000, n_side=1000)
Time: 0.456290 s (nthreads=1, npoints=1000000, n_side=1000)
Time: 0.455052 s (nthreads=1, npoints=1000000, n_side=1000)
Time: 0.449766 s (nthreads=1, npoints=1000000, n_side=1000)
Time: 0.458251 s (nthreads=1, npoints=1000000, n_side=1000)
Time: 0.447882 s (nthreads=1, npoints=1000000, n_side=1000)
Time: 0.449887 s (nthreads=1, npoints=1000000, n_side=1000)
Time: 0.473595 s (nthreads=1, npoints=1000000, n_side=1000)
```

Медиана = (0.452329 + 0.454839) / 2 = 0.453584 с

parallel_2_runs.txt:

```
Time: 0.241597 s (nthreads=2, npoints=1000000, n_side=1000)
Time: 0.244325 s (nthreads=2, npoints=1000000, n_side=1000)
Time: 0.247149 s (nthreads=2, npoints=1000000, n_side=1000)
Time: 0.242662 s (nthreads=2, npoints=1000000, n_side=1000)
Time: 0.242703 s (nthreads=2, npoints=1000000, n_side=1000)
Time: 0.239771 s (nthreads=2, npoints=1000000, n_side=1000)
Time: 0.239574 s (nthreads=2, npoints=1000000, n_side=1000)
Time: 0.247183 s (nthreads=2, npoints=1000000, n_side=1000)
Time: 0.239931 s (nthreads=2, npoints=1000000, n_side=1000)
Time: 0.244190 s (nthreads=2, npoints=1000000, n_side=1000)
```

Медиана = (0.242662 + 0.242703) / 2 = 0.242683 с

parallel_4_runs.txt:

```
Time: 0.140664 s (nthreads=4, npoints=1000000, n_side=1000)
Time: 0.140283 s (nthreads=4, npoints=1000000, n_side=1000)
Time: 0.139356 s (nthreads=4, npoints=1000000, n_side=1000)
Time: 0.136428 s (nthreads=4, npoints=1000000, n_side=1000)
Time: 0.138751 s (nthreads=4, npoints=1000000, n_side=1000)
Time: 0.138278 s (nthreads=4, npoints=1000000, n_side=1000)
Time: 0.138768 s (nthreads=4, npoints=1000000, n_side=1000)
```

```
Time: 0.140446 s (nthreads=4, npoints=1000000, n_side=1000)
Time: 0.141208 s (nthreads=4, npoints=1000000, n_side=1000)
Time: 0.140861 s (nthreads=4, npoints=1000000, n_side=1000)
```

Медиана = (0.139356 + 0.140283) / 2 = 0.139820 с

parallel_8_runs.txt:

```
Time: 0.094384 s (nthreads=8, npoints=1000000, n_side=1000)
Time: 0.092719 s (nthreads=8, npoints=1000000, n_side=1000)
Time: 0.093742 s (nthreads=8, npoints=1000000, n_side=1000)
Time: 0.095265 s (nthreads=8, npoints=1000000, n_side=1000)
Time: 0.095019 s (nthreads=8, npoints=1000000, n_side=1000)
Time: 0.089422 s (nthreads=8, npoints=1000000, n_side=1000)
Time: 0.097584 s (nthreads=8, npoints=1000000, n_side=1000)
Time: 0.095271 s (nthreads=8, npoints=1000000, n_side=1000)
Time: 0.093743 s (nthreads=8, npoints=1000000, n_side=1000)
Time: 0.091533 s (nthreads=8, npoints=1000000, n_side=1000)
```

Медиана = (0.093743 + 0.094384) / 2 = 0.094064 с

Для каждого варианта получили 10 значений времени и осталось вычислить медиану.

Конфигурация	Среднее, с	Медиана, с
sequential (без OpenMP)	0.443654	**0.442804**
OpenMP, p = 1	0.454786	**0.453584**
OpenMP, p = 2	0.242709	**0.242683**
OpenMP, p = 4	0.139504	**0.139820**
OpenMP, p = 8	0.093868	**0.094064**

Strong scaling (медианы, $\texttt{npoints} = 10^6$, `schedule(dynamic)`)

p	T(p), с (медиана)	S(p) = T(1)/T(p)	E(p) = S(p)/p
1	0.453584	1.000	1.000
2	0.242683	1.869	0.935
4	0.139820	3.244	0.811
8	0.094064	4.822	0.603

5. Графики и оценка

В файлах в директории лежит изображение с графиками, построенное в питоне на основе расчетов по данным от многократных вызовов выше.

График 1. Время от числа потоков

На графике показано время вычислений в зависимости от числа потоков. Данные взяты напрямую из замеров: для каждой конфигурации (p = 1, 2, 4, 8) по 10 запускам взята медиана времени.

Горизонтальная линия - это медиана по 10 запускам. По графику видно, что время падает с ростом p, не линейно и после перехода от 1 к 2 потокам прирост замедляется. Параллельная версия на одном потоке чуть медленнее последовательной, предположение: разница = кикие-то расходы OpenMP.

Как считалось: T(p) для каждой конфигурации — медиана 10 значений; T_seq — медиана 10 значений последовательной версии.

График 2. Ускорение против идеального

График показывает, во сколько раз быстрее стала программа при p потоках по сравнению с одним потоком. Идеальное ускорение, которого хотелось бы достичь - это прямая S(p) = p. Если бы каждый поток работал 100% времени по полной, время падало бы ровно в p раз.

Реальная линия ниже идеальной, и разрыв растёт с числом потоков, следовательно с увеличением p каждый дополнительный поток приносит всё меньше пользы.

Как считалось: S(p) = T(1) / T(p), где T(1) и T(p) - медианы 10 запусков, а идеал - S(p) = p.

График 3. Эффективность параллелизации

График показывает, какая доля от каждого потока реально используется, актуальность такого измерения возникла после предыдущего графика. Если E = 1, каждый поток работает на 100% (идеал), если E < 1, то часть времени потоки простаивают.

По графику видно, что при двух потоках эффективность ещё высокая, при четырёх - заметно падает, а при восьми каждый поток используется чуть больше половины времени. То есть чем больше потоков, тем меньше отдача от каждого. Вывод: для данного npoints увеличивать число потоков сверх четырёх невыгодно: прирост времени маленький, а эффективность падает.

Как считалось: E(p) = S(p) / p, где S(p) - ускорение из графика 2, p - число потоков.

График 4. Разброс времени по 10 запускам

На графике ящиков с усами для каждой конфигурации (seq, p = 1, 2, 4, 8) показан разброс времени по десяти запускам.

Коробки узкие, значит результаты стабильны, медиана хорошо совпадает со средним. Видны единичные выбросы (например, замедление при одном потоке и ускорение при восьми), но их мало, и на общую картину они не влияют.

Как считалось: для каждой конфигурации берутся 10 значений времени; коробка - 25-й и 75-й процентили, линия - медиана, усы - min/max без выбросов, точки - выбросы.

6. Таблица результатов

Внести медианное время по 10 запускам. Все времена должны быть измерены одним и тем же способом.

Версия	Число потоков (p)	Медианное время (T_p), с	Ускорение (S_p=T_1/T_p)	Эффективность (E_p=S_p/p)
Последовательная	1	**0,442804**	1,00	—
OpenMP	1	**0,453584**	**1,00**	**1,00**
OpenMP	2	**0,242683**	**1,87**	**0,935**
OpenMP	4	**0,139820**	**3,24**	**0,811**
OpenMP	8	**0,094064**	**4,82**	**0,603**

7. Результаты

Корректность

Результат сравнения CSV: файлы совпали

Что это означает: обе реализации работают одинаково корректно

Производительность

Наибольшее измеренное ускорение: 4,82 при 8 потоках.

Как меняется время при увеличении числа потоков:

при переходе от 1 к 2 потокам время падает почти вдвое: 0,4536 -> 0,2427 с

при переходе от 2 к 4 потокам время падает примерно в 1,74 раза: 0,2427 -> 0,1398 с

при переходе от 4 к 8 потокам время падает в 1,49 раза: 0,1398 -> 0,0941 с

То есть с ростом p абсолютный выигрыш от каждого следующего удвоения потоков уменьшается.

Интерпретация: Время уменьшается с ростом числа потоков, вычисления действительно распределяются между потоками, и до 4 потоков ускорение близко к линейному (эффективность 0,94 и 0,81).

Версия OpenMP с одним потоком медленнее последовательной, что может объясняться накладными расходами OpenMP.

8. Вывод

Сделаны последовательная и параллельная версии. При npoints = 1 000 000 результаты совпали побайтно (fc различий не нашёл, 167 492 строки), значит параллельная версия корректна. Максимальное ускорение 4,82 при 8 потоках. До 4 потоков масштабирование почти линейное (эффективность 0,94 и 0,81), на 8 падает до 0,60 из-за накладных расходов OpenMP,

неравномерной нагрузки и 6 физических ядер при 12 логических. Дальше 4 потоков наращивать невыгодно.